Perfect — you've pasted your full script, and what you want is a **step-by-step explanation** of how it works line by line and conceptually. Let's go through it in logical **phases**, so you actually understand what every part does and *why* it exists.

🧩 1️⃣ Imports and Environment Setup

```
from dotenv import load_dotenv
import os, numpy as np, csv, json
from sentence_transformers import SentenceTransformer
from elasticsearch import Elasticsearch
from dataclasses import dataclass
from groq import Groq
from typing import List, Dict, Any
```

Purpose:

- Load system utilities.
- Load `.env` variables into `os.environ`.
- Bring in the NLP embedding model (`SentenceTransformer`).
- Connect to **Elasticsearch** (the vector DB).
- Use **Groq API** (for justification).
- Use Python dataclasses to model `Startup` and `Investor` cleanly.

🧩 2️⃣ Environment Variables and Elasticsearch Client

```
load_dotenv()
ES_CLOUD_URL = os.getenv("ES_CLOUD_URL")
ES_API_KEY = os.getenv("ES_API_KEY")
GROQ_ENABLED = os.getenv("GROQ_ENABLED", "true").lower() in ("1","true","yes","y")

es = Elasticsearch(ES_CLOUD_URL, api_key=ES_API_KEY, verify_certs=True)
INVESTOR_INDEX = os.getenv("ES_INVESTOR_INDEX", "investors")
STARTUP_INDEX = os.getenv("ES_STARTUP_INDEX", "startups")
```

Meaning:

- Reads `.env` for credentials.
- Builds an **Elasticsearch client** that can connect either to:
  - Local: `http://localhost:9200`
  - Cloud: `https://yourcluster.elastic.cloud:443`
- Stores index names for both data types (configurable via `.env`).

- Sets up a flag (`GROQ_ENABLED`) to control if the LLM should run.

🧩 3️⃣ Load Sentence Transformer Model

```
model = SentenceTransformer("all-MiniLM-L6-v2")
embedding_dim = model.get_sentence_embedding_dimension()
MAX_RULE_SCORE = 140
JUSTIFICATION_SCORE_THRESHOLD = 50
```

Meaning:

- Loads a small BERT-like model that turns text → vector embeddings (384 dimensions).
- We'll use these for semantic similarity search.
- Cap rule-based scores at 140 (so we normalize properly).
- Only generate LLM justification if `final_score ≥ 50`.

🧩 4️⃣ Define Data Models

```
@dataclass
class Startup: ...
@dataclass
class Investor: ...
```

Meaning: Each object (row) in CSV is turned into a strongly typed object. This makes later operations like `startup.name` or `investor.portfolio_size` easy and reliable.

🧩 5️⃣ CSV Loaders

```
def load_startups_from_csv(filepath="startups.csv"):
    ...
def load_investors_from_csv(filepath="investors.csv"):
    ...
```

What happens:

- Opens CSV file
- Reads each row as a dictionary
- Builds a `Startup` or `Investor` dataclass object
- Returns a Python list of objects

☑️ Example:

```
startups = load_startups_from_csv()
investors = load_investors_from_csv()
```

Now you have clean Python objects with attributes like `.industry`, `.description`, etc.

🧩 6 Compute Embeddings

```
def compute_embedding(text: str) -> np.ndarray:
    emb = model.encode([text], normalize_embeddings=True)
    return np.array(emb).astype("float32")[0]
```

Meaning:

- Runs the sentence transformer on a given text (like a startup's description).
- Produces a 384-dimensional vector (numpy float32).
- Normalized so cosine similarity = dot product.

This is what Elasticsearch uses for **vector search**.

🧩 7 Rule-Based Scoring

```
def rule_based_score(startup: Startup, investor: Investor) -> float:
    score = 0.0
    ...
    normalized_score = (score / MAX_RULE_SCORE) * 100
```

Purpose: deterministic compatibility score (0–100).

Logic (and weights):

Factor	Description	Max Points
Industry overlap	Common industries	40
Stage match	Early/Seed/Series fit	30
Location same	City/state match	10
Check size fit	Funding goal within min/max	20
Team size fit	Fits investor's min/max	10
Revenue stage fit	Matches investor's preferences	10
Portfolio size	Scaled (max 10)	10
Experience years	Scaled (max 10)	10

Factor	Description	Max Points
Total possible raw score	140 → normalized to 100	—

This gives business-logic alignment before semantic similarity.

🧩 8️⃣ Elasticsearch Index Setup

```
def create_index(index_name: str):
    if es.indices.exists(index=index_name):
        es.indices.delete(index=index_name)
    mapping = {...}
    es.indices.create(index=index_name, body=mapping)
```

Meaning:

- Deletes existing index (fresh start)
- Creates a new index with:
  - `description_vector` (dense_vector, 384 dims)
  - `similarity: cosine`
  - `text fields`: id, name, description

🧩 9️⃣ Index Data

```
def index_startups(startups): ...
def index_investors(investors): ...
```

Each document contains:

```
{
  "id": "1",
  "name": "Fintech Innovations",
  "description": "...",
  "description_vector": [0.2, 0.1, ...]
}
```

After indexing, you call `es.indices.refresh()` to make it searchable.

🧩 🔍 1️⃣0️⃣ Vector Search Functions

```
def search_similar_investors(startup_emb, top_k=5): ...
def search_similar_startups(investor_emb, top_k=5): ...
```

They run an Elasticsearch KNN query:

```
{
  "knn": {
    "field": "description_vector",
    "query_vector": <embedding>,
    "k": 5,
    "num_candidates": 50
  }
}
```

→ Returns nearest documents by cosine similarity (semantic matches).

🧩 1⃣1⃣ LLM Justification

```
def get_llm_justification(investor, startup, rule_score, semantic_score,
final_score):
    client = Groq(api_key=GROQ_API_KEY)
    completion = client.chat.completions.create(
        model="meta-llama/llama-4-scout-17b-16e-instruct",
        messages=[{"role": "user", "content": prompt}]
    )
```

Purpose:

- Calls Groq's hosted Llama-4 model.
- Sends a detailed prompt (JSON-formatted data + analysis request).
- Gets a professional justification report.

If `GROQ_ENABLED=false`, this step is skipped (returns "no justification generated").

🧩 1⃣2⃣ Matching Functions

a) **Startups → Investors**

```
def match_startups_to_investors(top_k=5):
```

- Load both CSVs
- Create & index investors

- For each startup:
  - Get its embedding
  - Get candidate investors (rule > 0)
  - Vector search to get top semantic matches
  - Compute:

    ```
    semantic_score = ((hit["_score"] + 1)/2)*100
    final_score = 0.7 * rule_score + 0.3 * semantic_score
    ```

  - Generate justification if needed
  - Save result

---

b) **Investors → Startups**

Same idea, but reverse direction (investor query, startup index).

---

c) **Two-Way Merge**

```
def two_way_match(top_k=5):
    m1 = match_startups_to_investors(top_k)
    m2 = match_investors_to_startups(top_k)
    all_matches = m1 + m2
    all_matches.sort(...)
    return all_matches
```

☑ Now it's **bi-directional** matchmaking.

🧩 1⃣3⃣ Display and Interaction Layer (CLI)

Print top 50

```
def top_50_best_matches(matches):
    print table of Startup | Investor | Rule | Semantic | Final | Justification
    json.dump(matches, "matches_output.json")
```

Lookup Menus

Three utilities:

1. **Startup lookup** → see investors for a chosen startup
2. **Investor lookup** → see startups for a chosen investor
3. **Specific match** → directly check pair (name search)

They all re-use `matches` list and show formatted output.

🧩 1⃣4⃣ CLI Entry Point

```
if __name__ == "__main__":
    print("🚀 Starting Two-Way Hybrid Matching Engine...")
    matches = two_way_match(top_k=30)
    _ = top_50_best_matches(matches)
    ...
```

Then it runs an infinite menu loop:

- Lookup by investor
- Lookup by startup
- Check specific match
- Show top matches again
- Exit

🧩 1⃣5⃣ Full Runtime Flow

```
1⃣ Read .env → connect to Elasticsearch + Groq
2⃣ Load CSVs (startups + investors)
3⃣ Create + index investors
4⃣ Compute startup embeddings, match to investors
5⃣ Create + index startups
6⃣ Compute investor embeddings, match to startups
7⃣ Merge both results
8⃣ Compute final scores
9⃣ Generate LLM justifications (if enabled)
🔟 Display top 50, save JSON, and open CLI for drill-down
```

🧠 1⃣6⃣ Key Takeaways

Component	Purpose
`.env`	Switch between local & cloud easily
`dataclasses`	Structured, typed data
`sentence-transformers`	Embedding layer for semantic understanding

Component	Purpose
`Elasticsearch dense_vector`	Fast nearest-neighbor search
`rule_based_score()`	Deterministic business logic
`Groq LLM`	Generates human-readable explanations
`two_way_match()`	Symmetric matching (both directions)
CLI	Friendly way to explore matches

Would you like me to draw you a **step-by-step visual diagram** (with arrows & flow from .env → data → embeddings → ES → LLM → results) next? That'll make this even easier to remember.